OS ORANGE PROBLEM
SRN: PES1UG24CS640
NAME: VAIBHAV RAJ WARRIER
SECTION : 4-I

SCREENSHOT 1A)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$
```

SCREENSHOT 1B)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

SCREENSHOT 2A)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

SCREENSHOT 2B)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ ^C
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

SCREENSHOT 3A)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ make pes
rm -rf .pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
added: file1.txt
added: file2.txt
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  tree.c
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  tree.h
  untracked:  test_objects
  untracked:  commit.h
  untracked:  pes.h
  untracked:  pes.c
  untracked:  index.h
```

SCREENSHOT 3B)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776681040 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776681040 6 file2.txt
```

SCREENSHOT 4A)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ rm -rf .pes
./pes init

echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
added: hello.txt
Committed: e71d4d34cd98... Initial commit
added: hello.txt
Committed: 669cba0e0469... Add world
added: bye.txt
Committed: 9f984508268f... Add farewell
commit 9f984508268f0957f54ab83dc8551ae6989cb236a46332443b5b8665b1e1b7de
Author: PES User <pes@localhost>
Date:   1776681372

    Add farewell

commit 669cba0e046963c03ed016b2f574c84d17c0ef49573381122c81760a186cf33f
Author: PES User <pes@localhost>
Date:   1776681371

    Add world

commit e71d4d34cd98534b49b14a06f4091754e50904a609e5c2c167cb33a9c562dc4f
Author: PES User <pes@localhost>
Date:   1776681370

    Initial commit

vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$
```

SCREENSHOT 4B)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/66/9cba0e046963c03ed016b2f574c84d17c0ef49573381122c81760a186cf33f
.pes/objects/9f/984508268f0957f54ab83dc8551ae6989cb236a46332443b5b8665b1e1b7de
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/e7/1d4d34cd98534b49b14a06f4091754e50904a609e5c2c167cb33a9c562dc4f
.pes/refs/heads/main
```

SCREENSHOT 4C)

```
vboxuser@Ubuntu22:~/PES1UG24CS640-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
9f984508268f0957f54ab83dc8551ae6989cb236a46332443b5b8665b1e1b7de
ref: refs/heads/main
```

Phase 5 & 6: Analysis-Only Questions

The following questions cover filesystem concepts beyond the implementation scope of this lab. Answer them in writing — no code required.

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Analysis Answers

Q5.1

To implement pes checkout <branch>, I would do the following:

- Verify .pes/refs/heads/<branch> exists and read its commit hash.
- Update .pes/HEAD to ref: refs/heads/<branch> (unless supporting detached checkout explicitly).
- Resolve the target commit, read its root tree, and materialize that snapshot in the working directory:
  - Create missing directories.
  - Write tracked files from blob contents.
  - Remove tracked files/directories that are not present in the target tree.
- Rebuild .pes/index so it matches the checked-out tree (hash, mode, size, mtime metadata for tracked files).

The complexity comes from safe working-directory transitions: conflict detection, recursive tree expansion, path deletions, permission bits, and atomicity guarantees so a crash does not leave index/HEAD/working-tree inconsistent.

Q5.2

Dirty-check detection can be done with index + object store only:

- Read current HEAD commit and target branch commit.
- Expand both trees into path→blob-hash maps (and modes).
- For each path tracked in index:
  - Compare working file metadata (stat) with index metadata (mtime, size).
  - If metadata changed, hash the working file and compare to index hash to confirm real content change.
- A checkout conflict exists if:

i. File is dirty relative to index, and

ii. The file content/mode differs between current HEAD tree and target tree.

If both are true, refuse checkout for safety because switching would overwrite uncommitted local work.

Q5.3

In detached HEAD, HEAD stores a commit hash directly, not a branch ref. New commits still work, but they advance only HEAD (no branch name moves), so those commits become easy to lose once HEAD is moved away.

Recovery options:

- Immediately create a branch at that commit (refs/heads/<new-branch> = <commit-hash>).
- Or copy the hash from logs/reflog-like history and create a branch later.

- As long as GC has not deleted those unreachable commits, they remain recoverable by hash.

Q6.1

Mark-and-sweep GC algorithm:

1. **Roots:** all branch heads in .pes/refs/heads/* (and optionally HEAD if detached, tags if supported).
2. **Mark reachable objects:**
    - Traverse commit graph from each root using parent links.
    - For each commit, mark its tree.
    - Recursively walk trees; mark subtree/tree objects and blob objects.
3. **Sweep:**
    - Enumerate all object files in .pes/objects.
    - Delete objects not in the reachable set.

Best data structure: hash set of 32-byte object IDs (or 64-char hex IDs) for O(1) average membership checks.

For 100,000 commits and 50 branches, commit traversal is typically close to 100,000 unique commits (branches overlap heavily). Total visited objects are commits + all reachable trees + blobs. In a medium repository this is often a few hundred thousand to a few million objects.

Q6.2

Concurrent GC is dangerous because of a race:

- Commit process writes new blob/tree objects first.
- Before it writes the final commit and updates branch ref, those objects are temporarily unreachable from refs.
- GC runs during that window, sees those objects as unreachable, and deletes them.
- Commit then writes/updates refs to objects that no longer exist, corrupting history.

Git avoids this with multiple protections:

- Objects are first written as loose/temp files and made visible atomically.
- GC uses conservative reachability rules and grace periods via reflogs/prune windows.
- During packing/maintenance, Git relies on lockfiles and coordination to avoid deleting objects that may be referenced by in-flight operations.